1. Составляющие программирования
2. Базовый синтаксис и модель памяти Python
3. Присваивание и обмен значений переменных
4. Арифметические операции
5. Управляющие конструкции: циклы и ветвления

1. Составляющие программирования

Программирование как дисциплина включает в себя несколько ключевых аспектов. Изучение одного лишь синтаксиса языка (правил написания функций, блоков и операций) недостаточно для того, чтобы стать программистом. К основным составляющим относятся:

- Синтаксис языка программирования.
- Алгоритмы и структуры данных (основа решения вычислительных задач).
- Знание прикладных библиотек.
- Практика (ремесло) программирования — практические приемы и навыки написания чистого кода.
- Архитектура (дизайн) программного обеспечения — умение проектировать приложение в целом.
- Групповая работа — использование систем трекинга, планирования и взаимодействия в команде.

Основной упор в изучении фундаментального программирования делается на алгоритмы и структуры данных, так как они остаются неизменными десятилетиями, в отличие от быстро устаревающих прикладных библиотек.

2. Базовый синтаксис и модель памяти Python

Пустой файл уже является корректной программой на языке Python, которая ничего не делает. Для вывода информации на экран используется функция `print()` .

Пример:

```
print("Hello World")
```

В Python нет переменных в традиционном понимании (как выделенных ячеек памяти определенного типа). Вместо этого используются имена, которые ссылаются на объекты в памяти.

Пример:

```
x = "Hello World"
print(type(x))
```

В данном случае создается строковый объект `"Hello World"` типа `str`, и с ним связывается имя `x`. Оператор присваивания `=` в Python не копирует значение, а создает ссылку.

Тип данных привязан к самому объекту, а не к имени. Если имени присвоить результат нового вычисления, оно начнет ссылаться на новый объект.

Пример:

```
x = "Hello World"
x = 1 + 2 + 3 * 2 # Вычисляется выражение, создается объект 9 типа int
```

В этот момент старая связь теряется. Строковый объект `"Hello World"`, оставшийся без ссылок, автоматически удаляется из памяти механизмом, который называется сборщиком мусора.

3. Присваивание и обмен значений переменных

Существует несколько способов присваивания значений:

1. **Каскадное присваивание:** позволяет связать несколько имен с одним объектом.

   ```
   x = y = z = 0
   ```

2. **Множественное присваивание:** позволяет связать кортеж имен с кортежем значений.

   ```
   x, y, z = 1, 2, 3
   ```

Пример:
Обмен значений двух переменных.
Классический алгоритм обмена требует использования дополнительной (временной) переменной:

```
tmp = a
a = b
b = tmp
```

В Python благодаря множественному присваиванию обмен значений записывается лаконично:

```
a, b = b, a
```

При выполнении этой операции создается временный кортеж ссылок на объекты, происходит перекрестное связывание, после чего кортеж удаляется без копирования самих объектов.

Замечание: Также в языке присутствует синтаксический сахар — укороченные операторы присваивания, выполняющие арифметическое действие и связывание: `+=` , `-=` , `*=` , `/=` . Выражение `x += 1` эквивалентно `x = x + 1` .

4. Арифметические операции

Арифметические операции выполняются в порядке приоритета. Высший приоритет имеет возведение в степень, низший — сложение и вычитание.

1. **Возведение в степень:** x^y записывается как `x ** y` .
   - Допускается использование дробных степеней. Например, корень квадратный $\sqrt{3}$ записывается как `3 ** 0.5` .
   - Операция вычисляется справа налево. Выражение a^{b^c} запишется как `a ** b ** c` , и первой выполнится правая часть b^c.
2. **Унарный минус:** `-x` .
3. **Умножение и деление** (имеют одинаковый приоритет, вычисляются слева направо):
   - Умножение: `x * y` .
   - Деление: `x / y` . В результате всегда получается число с плавающей точкой (тип `float`), даже при делении целых чисел (например, $2/3$).
   - Целочисленное деление (с отбрасыванием остатка): `x // y` .
   - Взятие остатка от деления: `x % y` .

Замечание: Операции целочисленного деления и взятия остатка для отрицательных чисел в Python работают по строгим математическим правилам (остаток всегда положителен и принадлежит диапазону от 0 до делителя, не включая его). Это отличается от архитектурно ориентированных языков (C/C++).
Например:

$-12//5 = -3$
$-12 \pmod 5 = 3$ (так как $-3 \times 5 + 3 = -12$).

4. **Сложение и вычитание:** `x + y`, `x - y`.

5. Управляющие конструкции: циклы и ветвления

Блоки кода в Python выделяются исключительно отступами. Нет фигурных скобок или ключевых слов начала/конца блока.

Ветвления

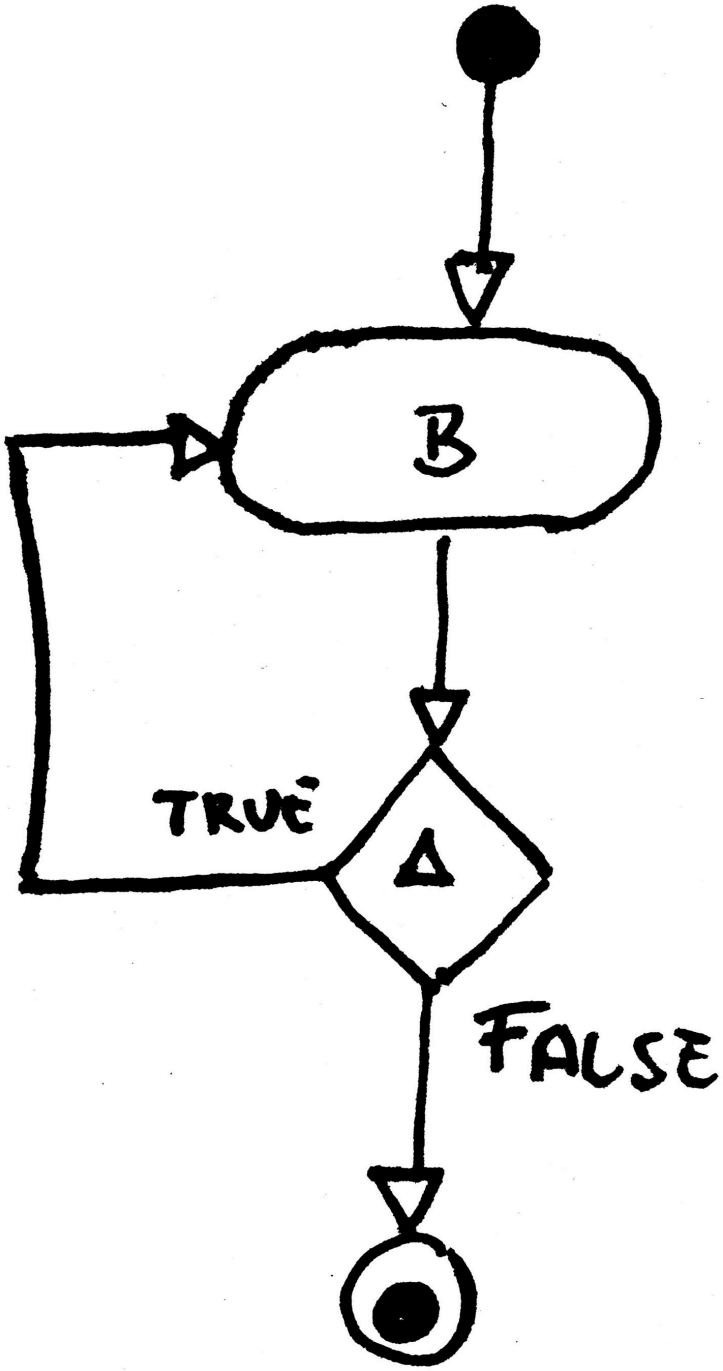
Условный оператор `if` позволяет выполнять код в зависимости от истинности условия.

```
if условие:
    оператор
else:
    оператор
```

Цикл while

Цикл `while` — это универсальный цикл с предусловием.

```
DO B
WHILE (A)
END WHILE
```



```
while условие:
    оператор 1 # Тело цикла
    оператор 2
else:
    оператор N # Выполняется после штатного завершения цикла
```

- **Итерация** — однократное выполнение тела цикла.
- Если условие ложно изначально, цикл не выполнится ни разу. Если условие всегда остается истинным (например, `while True:`), происходит зацикливание программы.
- Блок `else` выполняется один раз после завершения всех итераций, когда условие становится ложным.

Дополнительные инструкции управления циклом:

- `break` : экстренное прерывание цикла. При вызове `break` программа выходит из цикла, минуя выполнение блока `else` .
- `continue` : досрочный переход к следующей итерации. Оставшаяся часть тела цикла пропускается, и происходит немедленный переход к проверке условия цикла.

Допускается вложенность циклов: внутри тела одного цикла может находиться другой цикл.

Цикл for

Цикл `for` используется для итерации по последовательностям.
Для генерации арифметических прогрессий используется функция `range` . Она принимает три аргумента: `Start` (начало, включительно), `Stop` (конец, не включительно) и `Step` (шаг).

Пример:

```
for x in range(1, 10, 1):
    print(x)
```

Данный код сгенерирует числа от 1 до 9.
Если задать отрицательный шаг, необходимо, чтобы начальное значение было больше конечного (например, `range(10, 1, -1)` сгенерирует последовательность от 10 до 2). Если задать невыполнимые условия (шаг 1, начало 10, конец 1), будет сгенерирована пустая последовательность, и тело цикла не выполнится ни разу. В цикле `for` также применимы инструкции `break` , `continue` и блок `else` .